operating system will automatically save the state and switch to task B or C when appropriate, and if the target system has multiple cores or processors, tasks A and B may well be able to run truly concurrently. The code for handling the key press or network packet will now be run in a timely fashion, and everybody wins: the user gets timely responses, and you as developer have simpler code because each thread can focus on doing operations related directly to its responsibilities, rather than getting mixed up with control flow and user interaction.

That sounds like a nice, rosy vision. Can it really be like that? As with everything, it depends on the details. If everything really is independent, and the threads have no need to communicate with each other, then it really is this easy. Unfortunately, the world is rarely like that. These nice background tasks are often doing something that the user requested, and they need to let the user know when they're done by updating the user interface in some manner. Alternatively, the user might want to cancel the task, which therefore requires the user interface to somehow send a message to the background task telling it to stop. Both these cases require careful thought and design and suitable synchronization, but the concerns are still separate. The user interface thread still just handles the user interface, but it might have to update it when asked to do so by other threads. Likewise, the thread running the background task still just focuses on the operations required for that task; it just happens that one of them is "allow task to be stopped by another thread." In neither case do the threads care where the request came from, only that it was intended for them and relates directly to their responsibilities.

There are two big dangers with separating concerns with multiple threads. The first is that you'll end up separating the *wrong* concerns. The symptoms to check for are that there is a lot of data shared between the threads or the different threads end up waiting for each other; both cases boil down to too much communication between threads. If this happens, it's worth looking at the reasons for the communication. If all the communication relates to the same issue, maybe that should be the key responsibility of a single thread and extracted from all the threads that refer to it. Alternatively, if two threads are communicating a lot with each other but much less with other threads, maybe they should be combined into a single thread.